IoT 1

Les 5– Structs, Pointers, Alternate Functions, USART, …

Struct

Registers bundelen in Structs

- **Wat** is een struct?
  - Een `struct` is een ‘**complex**’ **datatype**.
  - Het definieert variabelen die **samen één geheel** vormen.
  - Je kan alle variabelen bereiken **via één naam**… Handig!

- **Voordeel**?
  - **Struct**uur…! ;-)
  - Kan **via header files toegevoegd** worden aan je project.

Registers bundelen in Structs

- **Voorbeeld**

```
1  // Is de struct Person nog onbekend?
2  // OPM: een '#" betekent dat er een 'preprocessor directive' komt...
3  #if !defined(PERSON_DEFINED)
4    // Definieer die nu, zodat die maar één keer gedefinieerd wordt in totaal!
5    #define PERSON_DEFINED
6
7    // De opbouw van de struct bepalen en een naam kiezen.
8    typedef struct
9    {
10     // Tekst met plaats voor 50 karakters + nul karakter.
11     // Bijvoorbeeld: "Kathy\0".
12     char firstName[51];
13     char lastName[51];
14
15     // Plaats voor het opslaan van een kommagetal.
16     float weight;
17   }Person;
18 #endif
```

Merk op: '\0' is een '**escape sequence**'. Net als '\n' of '\r'... Je 'ontsnapt' aan de normale gang van zaken...

hogeschool vives

Registers bundelen in Structs

- **Voorbeeld**

```
10 #include "usart2.h"
11 #include "person.h"
12
13 // Functie prototypes.
14 void SystemClock_Config(void);
15 void InitIo(void);
16 void WaitForMs(uint32_t timespan);
17 void PersoonToUsart(Person * person);
18
19 // Variabelen aanmaken.
20 char text[101];
21 uint8_t count = 0;
22 volatile uint32_t ticks = 0;
23 Person bigGuy;
```

Voeg een **verwijzing** naar de '**bibliotheek**' toe.

Maak een nieuwe variabele van het nieuwe type *Person*.

Registers bundelen in Structs

- **Voorbeeld**

```
Reboot
John Beton (123.45 kg)
```

```c
// Struct opvullen met info.
sprintf(bigGuy.firstName, "John");
sprintf(bigGuy.lastName, "Beton");
bigGuy.weight = 123.45f;

// Optie 1: namen en gewicht zonder pointers tonen aan de gebruiker.
StringToUsart2(bigGuy.firstName);
StringToUsart2(" ");
StringToUsart2(bigGuy.lastName);
StringToUsart2(" (");
sprintf(text, "%.2f", bigGuy.weight);
StringToUsart2(text);
StringToUsart2(" kg)\r\n");
```

Een ***float*** **omzetten** naar een tekst/string, doe je via *%f*.

Pointers

hogeschool vives

Pointers

- **Wat**?
  - **Variabelen** in C-taal, worden **bewaard** op een bepaalde locatie **in het RAM-geheugen**.
  - Die **locaties** kan je bereiken **via een 'adres'**.
  - Zo'n adres is in feite niets anders dan **een getal** dat naar een locatie verwijst.
  - To point = **verwijzen**…
  - Er bestaan in C-taal, **pointers naar variabelen**…
  - **Super belangrijk** als je goed aan 'embedded programming' wil doen!

Pointers

- **Hoe**?

Op adres 12543 'woont' de variabele met naam '*getal*'. De inhoud van die variabele heeft de waarde 138.

Hier werken we **zonder pointers**.

In **C-taal** wordt dit:

```
uint8_t getal;
getal = 138;
```

Grafische voorstelling van het RAM-geheugen

adres: 12540	naam: ?	adres: 12640	naam: ?
inhoud:	?	inhoud:	?
adres: 12541	naam: ?	adres: 12641	naam: ?
inhoud:	?	inhoud:	?
adres: 12542	naam: ?	adres: 12642	naam: ?
inhoud:	?	inhoud:	?
adres: 12543	naam: getal	adres: 12643	naam: ?
inhoud:	138	inhoud:	?
adres: 12544	naam: ?	adres: 12644	naam: ?
inhoud:	?	inhoud:	?
...		...	
adres: 12600	naam: ?	adres: 12700	naam: ?
inhoud:	?	inhoud:	?

hogeschool vives

Pointers

- **Hoe**?

Op adres 12641 'woont' de variabele met naam '*getalPointer*'. De inhoud van die variabele kan een **adreswaarde** bevatten.

Hier werken we dus **mét pointers**.

Gebruik daarvoor de asterisk (*)…

In **C-taal** wordt dit:
uint8_t getalPointer;*

Grafische voorstelling van het RAM-geheugen

adres: 12540	naam: ?	adres: 12640	naam: ?
inhoud: ?		inhoud: ?	
adres: 12541	naam: ?	adres: 12641	naam: getalPointer
inhoud: ?		inhoud: ?	
adres: 12542	naam: ?	adres: 12642	naam: ?
inhoud: ?		inhoud: ?	
adres: 12543	naam: getal	adres: 12643	naam: ?
inhoud: 138		inhoud: ?	
adres: 12544	naam: ?	adres: 12644	naam: ?
inhoud: ?		inhoud: ?	
…		…	
adres: 12600	naam: ?	adres: 12700	naam: ?
inhoud: ?		inhoud: ?	

Pointers

- **Hoe**?

Het **adres** van de variabele met naam '*getal*', kan je ook **opvragen** in C-taal. Dat doe je via een 'ampersand' (**&**).

Dat adres kan je **bewaren in een pointer** naar een variabele (van het juiste type).

In **C-taal** wordt dit:
… = &getal;

Grafische voorstelling van het RAM-geheugen

adres: 12540	naam: ?	adres: 12640	naam: ?
inhoud: ?		inhoud: ?	
adres: 12541	naam: ?	adres: 12641	naam: getalPointer
inhoud: ?		inhoud: ?	
adres: 12542	naam: ?	adres: 12642	naam: ?
inhoud: ?		inhoud: ?	
adres: 12543	naam: getal	adres: 12643	naam: ?
inhoud: 138		inhoud: ?	
adres: 12544	naam: ?	adres: 12644	naam: ?
inhoud: ?		inhoud: ?	
…		…	
adres: 12600	naam: ?	adres: 12700	naam: ?
inhoud: ?		inhoud: ?	

Pointers

- **Hoe**?

Op adres 12641 'woont' de variabele met naam '*getalPointer*'. De inhoud van die variabele krijgt de adreswaarde 12543.

Hier werken we dus **mét pointers**.

In **C-taal** wordt dit:
getalPointer = &getal;

Grafische voorstelling van het RAM-geheugen

adres: 12540	naam: ?	adres: 12640	naam: ?
inhoud: ?		inhoud: ?	
adres: 12541	naam: ?	adres: 12641	naam: getalPointer
inhoud: ?		inhoud: 12543	
adres: 12542	naam: ?	adres: 12642	naam: ?
inhoud: ?		inhoud: ?	
adres: 12543	naam: getal	adres: 12643	naam: ?
inhoud: 138		inhoud: ?	
adres: 12544	naam: ?	adres: 12644	naam: ?
inhoud: ?		inhoud: ?	
...		...	
adres: 12600	naam: ?	adres: 12700	naam: ?
inhoud: ?		inhoud: ?	

Pointers

- **Hoe**?

Via `getalPointer` kan je dus ook `getal` bereiken. Via een ‘verwijzing/pointer’…

Gebruik daarvoor **ook** de asterisk (*)…

In **C-taal** wordt dit:
`… = *getalPointer;`

Grafische voorstelling van het RAM-geheugen

adres: 12540	naam: ?	adres: 12640	naam: ?
inhoud: ?		inhoud: ?	
adres: 12541	naam: ?	adres: 12641	naam: getalPointer
inhoud: ?		inhoud: 12543	
adres: 12542	naam: ?	adres: 12642	naam: ?
inhoud: ?		inhoud: ?	
adres: 12543	naam: getal	adres: 12643	naam: ?
inhoud: 138		inhoud: ?	
adres: 12544	naam: ?	adres: 12644	naam: ?
inhoud: ?		inhoud: ?	
…		…	
adres: 12600	naam: ?	adres: 12700	naam: ?
inhoud: ?		inhoud: ?	

Pointers

- **Let dus op**!
  - Het asterisk-teken, heeft twee betekenissen!
    1. Om een pointer naar een **variabele aan te maken**.

       ```
       uint8_t* getalPointer;
       ```

    2. Om de **inhoud** waarnaar een pointer verwijst, **op te vragen**.

       ```
       … = *getalPointer;
       ```


hogeschool vives

Pointers

- **Voorbeeld 1**

```
// Variabelen aanmaken.
static uint8_t count = 0, getal = 138;
static uint8_t* getalPointer;
static volatile uint32_t ticks = 0;
static char text[101];
```

Pointers

- **Voorbeeld 1**

```
47      sprintf(text, "Waarde van getal: %d.\r\n", getal);
48      StringToUsart2(text);
49
50      sprintf(text, "Adres van getal: %d.\r\n", (uint32_t)&getal);
51      StringToUsart2(text);
52
53      getalPointer = &getal;
54
55      sprintf(text, "Waarde van getalPointer: %d.\r\n", (uint32_t)getalPointer);
56      StringToUsart2(text);
57
58      sprintf(text, "Adres van getalPointer: %d.\r\n", (uint32_t)&getalPointer);
59      StringToUsart2(text);
60
61      sprintf(text, "Waarde naar waar getalPointer wijst: %d.\r\n", *getalPointer);
62      StringToUsart2(text);
63
64      WaitForMs(5000);
65
66      // Pas de waarde van getal aan via getalPointer...
67      *getalPointer = 139;
```

Pointers

- **Voorbeeld 1**

$(138)_{10} = (8A)_{hex}$

$(536870916)_{10} = (20000004)_{hex}$

$(536871016)_{10} = (20000068)_{hex}$

COM11 - PuTTY

```
Reboot
Waarde van getal: 138.
Adres van getal: 536870916.
Waarde van getalPointer: 536870916.
Adres van getalPointer: 536871016.
Waarde naar waar getalPointer wijst: 138.
Waarde van getal: 139.
Adres van getal: 536870916.
Waarde van getalPointer: 536870916.
Adres van getalPointer: 536871016.
Waarde naar waar getalPointer wijst: 139.
```


hogeschool vives

Pointers

- **Voorbeeld 1**

Kan je **zelf** in debug mode deze **Memory vensters oproepen?**

$(536870916)_{10} = (20000004)_{hex}$

$(138)_{10} = (8A)_{hex}$

Memory 1

Address: 536870916

```
0x20000004: 8A 00 00 00 00 00 00 00 00
0x2000003F: 00 00 00 00 00 00 00 00 00
0x2000007A: 6C 3A 20 35 33 36 38 37 30
0x200000B5: 00 00 00 00 00 00 00 00 00
0x200000F0: 00 00 00 00 00 00 00 00 00
```

$(536871016)_{10} = (20000068)_{hex}$

Memory 1

Address: 536871016

```
0x20000068: 04 00 00 20 41 64 72 65 73 20
0x200000A3: 00 00 00 00 00 00 00 00 00 00
0x200000DE: 00 00 00 00 00 00 00 00 00 00
0x20000119: 00 00 00 00 00 00 00 00 00 00
0x20000154: 00 00 00 00 00 00 00 00 00 00
```

Pointers

- **Voorbeeld 2**
  - We grijpen terug naar het voorbeeld i.v.m. structs… Maar gebruiken nu ook **pointers naar structs**.

```
51  // Optie 2: namen en gewicht via een functie en een pointer naar een struct
52  //          tonen aan de gebruiken.
53  PersoonToUsart(&bigGuy);
54
55  // Enkele lege lijnen invoegen.
56  StringToUsart2("\r\n\r\n");
```

COM11 - PuTTY

```
Reboot
John Beton (123.45 kg)
John Beton (123.45 kg)
John Beton (123.45 kg)
```

Pointers

- **Voorbeeld 2**

```
65 // Maak een nieuwe functie om een 'Person' naar de USART te sturen.
66 // De functie neemt een 'pointer naar een Person' mee.
67 void PersoonToUsart(Person * person)
68 {
69   StringToUsart2(person->firstName);
70   StringToUsart2(" ");
71   StringToUsart2(person->lastName);
72   StringToUsart2(" (");
73   sprintf(text, "%.2f", person->weight);
74   StringToUsart2(text);
75   StringToUsart2(" kg)\r\n");
76
77   // of
78
79   sprintf(text, "%s %s (%.2f kg)", person->firstName, person->lastName, person->weight);
80   StringToUsart2(text);
81 }
```

Voordeel?

Doorgeven van **parameters** gaat **veel efficiënter**!

Je hoeft slechts één adres te kopiëren i.p.v. alle data van de gehele struct!

Nadeel?

hogeschool vives

Pointers

- **Waar nog?**
  - We hebben **al heel veel structs en pointers gebruikt** tot nu toe, maar dat viel misschien niet op…
  - **Bekijk** maar eens statements zoals hieronder!

```c
void InitLeds(void)
{
  // Clock voor GPIOA inschakelen.
  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
```

main.c | person.h | leds.c | stm32f091xc.h

```c
#define OB                    ((OB_TypeDef *) OB_BASE)
#define RCC                   ((RCC_TypeDef *) RCC_BASE)
#define CRC                   ((CRC_TypeDef *) CRC_BASE)
```


hogeschool vives

Pointers

- **Waar nog?**

```
463 typedef struct
464 {
465   __IO uint32_t CR;           /*!< RCC clock control register,
466   __IO uint32_t CFGR;       /*!< RCC clock configuration register,
467   __IO uint32_t CIR;        /*!< RCC clock interrupt register,
468   __IO uint32_t APB2RSTR;   /*!< RCC APB2 peripheral reset register,
469   __IO uint32_t APB1RSTR;   /*!< RCC APB1 peripheral reset register,
470   __IO uint32_t AHBENR;     /*!< RCC AHB peripheral clock register,
471   __IO uint32_t APB2ENR;    /*!< RCC APB2 peripheral clock enable register,
472   __IO uint32_t APB1ENR;    /*!< RCC APB1 peripheral clock enable register,
473   __IO uint32_t BDCR;       /*!< RCC Backup domain control register,
474   __IO uint32_t CSR;        /*!< RCC clock control & status register,
475   __IO uint32_t AHBRSTR;    /*!< RCC AHB peripheral reset register,
476   __IO uint32_t CFGR2;      /*!< RCC clock configuration register 2,
477   __IO uint32_t CFGR3;      /*!< RCC clock configuration register 3,
478   __IO uint32_t CR2;        /*!< RCC clock control register 2,
479 } RCC_TypeDef;
```


hogeschool vives

Alternate functions

Alternate functions

- **Wat**?
  - Is letterlijk een ‘**alternatieve functie**’ op een **pin** van de microcontroller.
  - Standaard zijn vier mogelijkheden:
    1. digitale ingang
    2. digitale uitgang
    3. **alternatieve functie**
    4. analoge in- of uitgang
  - Zie GPIOx_**MODER** in de reference manual…

Alternate functions

- **Mogelijkheden**?
  - De verschillende mogelijke werkingen per pin, kan je opzoeken in de '**device specifieke datasheet**'.
  - Merk op: dat is dus niet in de 'reference manual'.
  - Bekijk op de volgende slides en/of de device specifieke datasheet:
    - welk soort **I/O structure** de pin heeft.
    - welke **alternate functions** de pin heeft.
    - de **overzichtstabel** met alternate functions.
  - Neem als **voorbeeld PA0** als **USART4 TX**…

Alternate functions

Table 12. Legend/abbreviations used in the pinout table

Name		Abbreviation	Definition
Pin name		Unless otherwise specified in brackets below the pin name, the pin function during and after reset is the same as the actual pin name	
Pin type		S	Supply pin
		I/O	Input / output pin
I/O structure		FT	5 V-tolerant I/O
		FTf	5 V-tolerant I/O, FM+ capable
		TTa	3.3 V-tolerant I/O directly connected to ADC
		TC	Standard 3.3 V I/O
		RST	Bidirectional reset pin with embedded weak pull-up resistor
Notes		Unless otherwise specified by a note, all I/Os are set as floating inputs during and after reset.	
Pin functions	Alternate functions	Functions selected through GPIOx_AFR registers	
	Additional functions	Functions directly selected/enabled through peripheral registers	

Bron: STM32F091xC datasheet (st.com)

hogeschool vives

Alternate functions

Table 13. STM32F091xB/xC pin definitions (continued)

Pin numbers						Pin name (function upon reset)	Pin type	I/O structure	Notes	Pin functions	
UFBGA100	LQFP100	UFBGA64	LQFP64	WLCSP64	LQFP48/UFQFPN48					Alternate functions	Additional functions
J1	19	-	-	-	-	PF2	I/O	FT		EVENTOUT, USART7_TX, USART7_CK_RTS	WKUP8
K1	20	F1	12	G8	8	VSSA	S	-		Analog ground	
M1	21	H1	13	H8	9	VDDA	S	-		Analog power supply	
L1	22	-	-	-	-	PF3	I/O	FT		EVENTOUT, USART7_RX, USART6_CK_RTS	
L2	23	G2	14	F7	10	PA0	I/O	TTa		USART2_CTS, TIM2_CH1_ETR, TSC_G1_IO1, USART4_TX COMP1_OUT	RTC_ TAMP2, WKUP1, ADC_IN0, COMP1_INM6

Bron: STM32F091xC datasheet (st.com)

Alternate functions

Table 14. Alternate functions selected through GPIOA_AFR registers for port A

Pin name	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
PA0	-	USART2_CTS	TIM2_CH1_ETR	TSC_G1_IO1	USART4_TX	-	-	COMP1_OUT
PA1	EVENTOUT	USART2_RTS	TIM2_CH2	TSC_G1_IO2	USART4_RX	TIM15_CH1N	-	-
PA2	TIM15_CH1	USART2_TX	TIM2_CH3	TSC_G1_IO3	-	-	-	COMP2_OUT
PA3	TIM15_CH2	USART2_RX	TIM2_CH4	TSC_G1_IO4	-	-	-	-
PA4	SPI1_NSS, I2S1_WS	USART2_CK	-	TSC_G2_IO1	TIM14_CH1	USART6_TX	-	-
PA5	SPI1_SCK, I2S1_CK	CEC	TIM2_CH1_ETR	TSC_G2_IO2	-	USART6_RX	-	-
PA6	SPI1_MISO, I2S1_MCK	TIM3_CH1	TIM1_BKIN	TSC_G2_IO3	USART3_CTS	TIM16_CH1	EVENTOUT	COMP1_OUT
PA7	SPI1_MOSI, I2S1_SD	TIM3_CH2	TIM1_CH1N	TSC_G2_IO4	TIM14_CH1	TIM17_CH1	EVENTOUT	COMP2_OUT
PA8	MCO	USART1_CK	TIM1_CH1	EVENTOUT	CRS_SYNC	-	-	-
PA9	TIM15_BKIN	USART1_TX	TIM1_CH2	TSC_G4_IO1	I2C1_SCL	MCO	-	-
PA10	TIM17_BKIN	USART1_RX	TIM1_CH3	TSC_G4_IO2	I2C1_SDA	-	-	-
PA11	EVENTOUT	USART1_CTS	TIM1_CH4	TSC_G4_IO3	CAN_RX	I2C2_SCL	-	COMP1_OUT
PA12	EVENTOUT	USART1_RTS	TIM1_ETR	TSC_G4_IO4	CAN_TX	I2C2_SDA	-	COMP2_OUT
PA13	SWDIO	IR_OUT	-	-	-	-	-	-
PA14	SWCLK	USART2_TX	-	-	-	-	-	-
PA15	SPI1_NSS, I2S1_WS	USART2_RX	TIM2_CH1_ETR	EVENTOUT	USART4_RTS	-	-	-

Bron: STM32F091xC datasheet (st.com)

hogeschool vives

Alternate functions

- **Hoe**?
  - Stel de **GPIOx_AFRL** of **GPIOx_AFRH** registers in, zodat de juiste alternate function op de juiste pin komt.

8.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..F)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFSEL7[3:0]				AFSEL6[3:0]				AFSEL5[3:0]				AFSEL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFSEL3[3:0]				AFSEL2[3:0]				AFSEL1[3:0]				AFSEL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	0	1	0	0

Bits 31:0 **AFSELy[3:0]:** Alternate function selection for port x pin y (y = 0..7)
These bits are written by software to configure alternate function I/Os

AFSELy selection:

0000: AF0
0001: AF1
0010: AF2
0011: AF3
0100: AF4
0101: AF5
0110: AF6
0111: AF7
1000: Reserved
1001: Reserved
1010: Reserved
1011: Reserved
1100: Reserved
1101: Reserved
1110: Reserved
1111: Reserved

Bron: STM32F0 Reference Manual – RM0091 (st.com)

USART(2)

USART

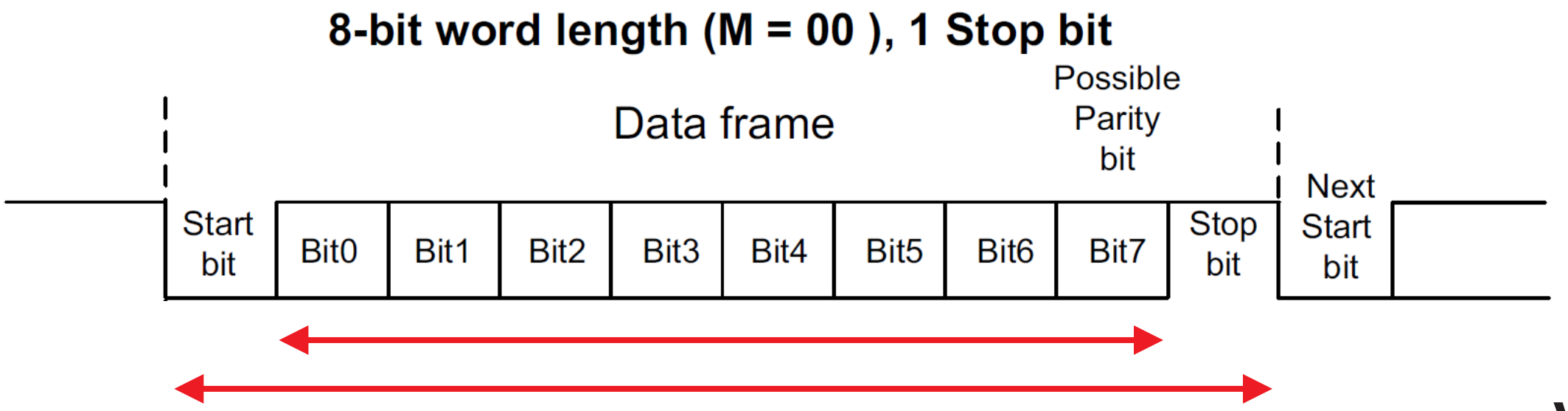
- **Wat**?
  - UART = **U**niversal **A**synchronous **R**eceiver **T**ransmitter
  - USART = **U**niversal **S**ynchronous **A**synchronous **R**eceiver **T**ransmitter
  - synchroon <-> asynchroon?!

    De module kan ook synchroon gebruikt worden, maar dit valt buiten het bestek van deze cursus.

  - Ook bekend als: RS-232, **RS-485** (bij DMX), …

UART

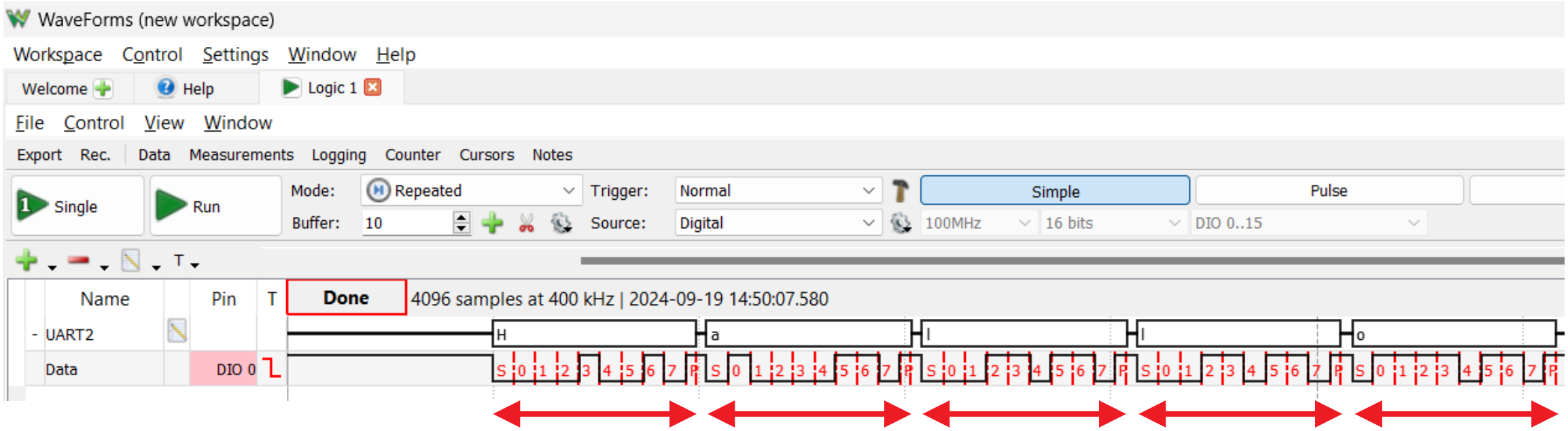
- UART **dataframe**
  - Typisch frame bestaat uit **8-bit**, met **start-** en **stopbits**.
  - Spreek ook een datatransfertsnelheid af, de '**baudrate**'.
  - Data kan verstuurd worden via **één draadje**… TX -> RX.



Bron: STM32F0 Reference Manual – RM0091 (st.com)

UART

- UART **dataframe**
  - **Meting** met USB-oscilloscoop.
  - De microcontroller verzendt de tekst “**Hallo**”.



UART

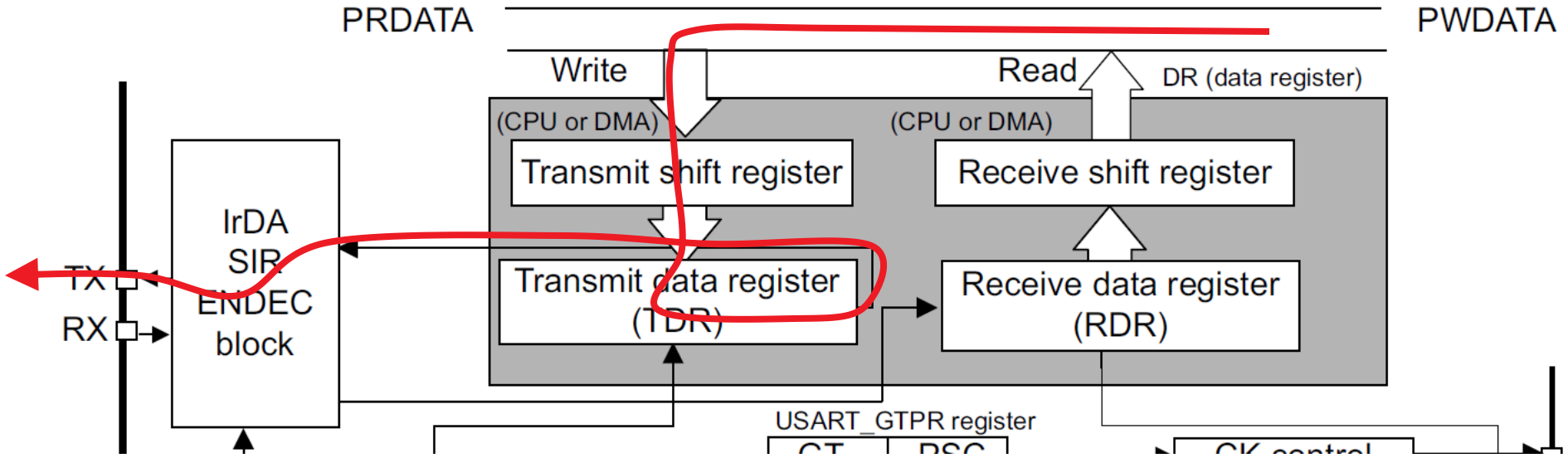
- **Nucleo** en UART
  - Ons Nucleo-bord is gekoppeld met **UART2 via USB** (ST-link programmer).
  - Daarvoor worden **PA2** en **PA3** gebruikt.

  - De USB-poort **maakt daarbij een 'virtuele seriële poort'** aan. Daardoor **lijkt het alsof** er een 'gewone seriële poort' verbonden is aan je computer.

UART

- **Blokschema**

Kan je de **weg volgen** die de data aflegt om **verstuurd** te worden vanuit de microcontroller **naar een ander apparaat**?

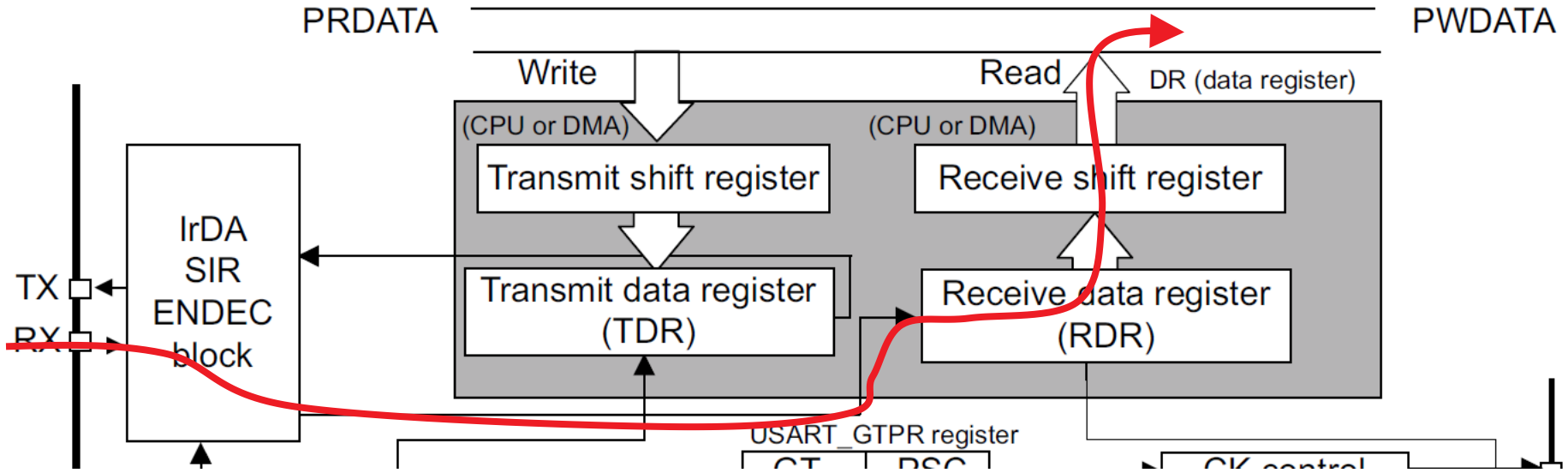


Bron: STM32F0 Reference Manual – RM0091 (st.com)

UART

- **Blokschema**

Kan je de **weg volgen** die de data aflegt om **ontvangen** te worden in de microcontroller **vanuit een ander apparaat**?



Bron: STM32F0 Reference Manual – RM0091 (st.com)

UART

- **Instellingen** voor het gebruik van UART's

1. De gebruikte poort van **klok** voorzien.
2. De gebruikte pinnen op **alternate function** zetten.
3. De alternate **function kiezen**.
4. De UART **initialiseren** met de gewenste parameters (**baud** rate, aantal stopbits, ...).

UART

- **Instellingen** voor het gebruik van UART's

Bekijk de volledige code in '**usart2.c**'!

```c
void InitUsart2(uint32_t baudRate)
{
  // Clock voor GPIOA inschakelen.
  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;

  // Pinnen van de USART2 instellen als alternate function.
  // PA2 => TX-pin
  GPIOA->MODER = (GPIOA->MODER & ~GPIO_MODER_MODER2) | GPIO_MODER_MODER2_1;
  GPIOA->AFR[0] |= 0x00000100;

  // PA3 => RX-pin
  GPIOA->MODER = (GPIOA->MODER & ~GPIO_MODER_MODER3) | GPIO_MODER_MODER3_1;
  GPIOA->AFR[0] |= 0x00001000;

  // Usart module van een klok voorzien
  RCC->APB1ENR |= RCC_APB1ENR_USART2EN;

  // Baudrate zetten
  USART2->BRR = 48000000/baudRate;
```


hogeschool vives

UART

- **Complexiteit** versus efficiëntie
  - Zowel voor het versturen als voor het ontvangen van data heb je de keuze tussen het werken via:

→ **Polling** (eenvoudige code)

of

→ **Interrupts**

of

DMA (complexere code)

hogeschool vives

Toepassingsvoorbeeld: communicatie COM-poort via **polling**

```
37    // Oneindige lus starten.
38    while (1)
39    {
40      // Even wachten.
41      WaitForMs(2000);
42
43      // User LED (onboard) toggle'n... (PA5)
44      GPIOA->ODR = GPIOA->ODR ^ GPIO_ODR_5;
45
46      // Teken van leven geven.
47      StringToUsart2("Hallo.... \r\n");
48
49      // Is er een een byte ontvangen via de USART2?
50      // Lees de info in via 'polling' (= op gezette momenten vragen of er info is).
51      if((USART2->ISR & USART_ISR_RXNE) == USART_ISR_RXNE)
52      {
53        ByteToLeds((uint8_t)USART2->RDR);
54      }
55    }
56  }
57
```

Voorzie je hier **problemen** bij het ontvangen van data?

Test via Putty.

hogeschool vives

Toepassingsvoorbeeld: communicatie COM-poort via **interrupts**

```
34    // Interrupt voor de receiver enable
35    // (receiver buffer not empty interrupt enable)
36    USART2->CR1 |= USART_CR1_RXNEIE;
37
38    // Koppeling van interrupt maken met de NVIC
39    NVIC_SetPriority(USART2_IRQn, 0);
40    NVIC_EnableIRQ(USART2_IRQn);
41
42    // Receiver enable
43    USART2->CR1 |= USART_CR1_RE;
44  }
```

Voorzie je hier **problemen** bij het ontvangen van data?

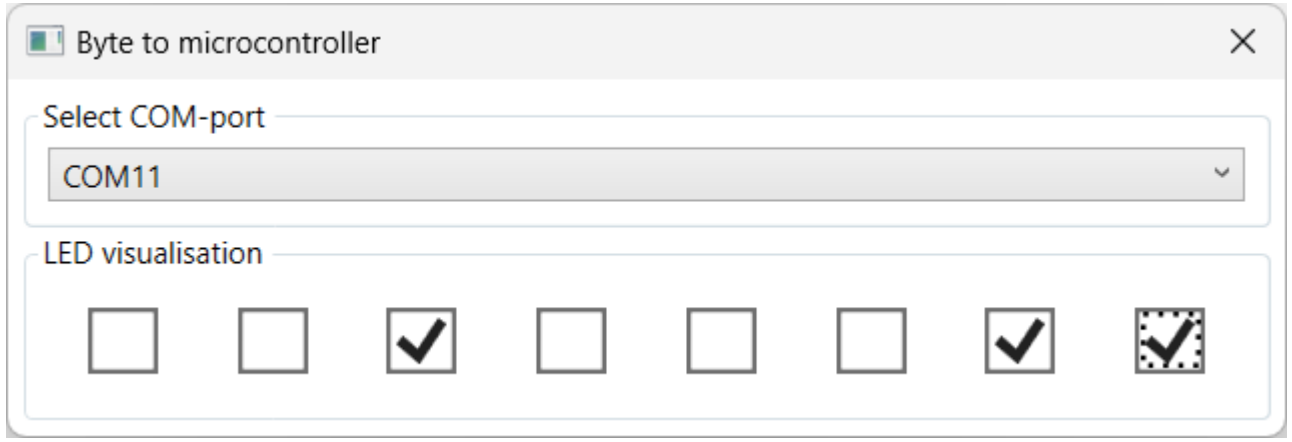
Test via Putty.

```
51  // Lees de info in via 'interrupts' (= op moment van ontvangen de data inlezen).
52  void USART2_IRQHandler(void)
53  {
54    if((USART2->ISR & USART_ISR_RXNE) == USART_ISR_RXNE)
55    {
56      // Byte ontvangen, lees hem om alle vlaggen te wissen.
57      uint8_t temp = USART2->RDR;
58
59      ByteToLeds(temp);
60    }
61  }
```

Byte to Microcontroller

- Kan je **vanuit C#** één byte versturen **naar je microcontroller**?

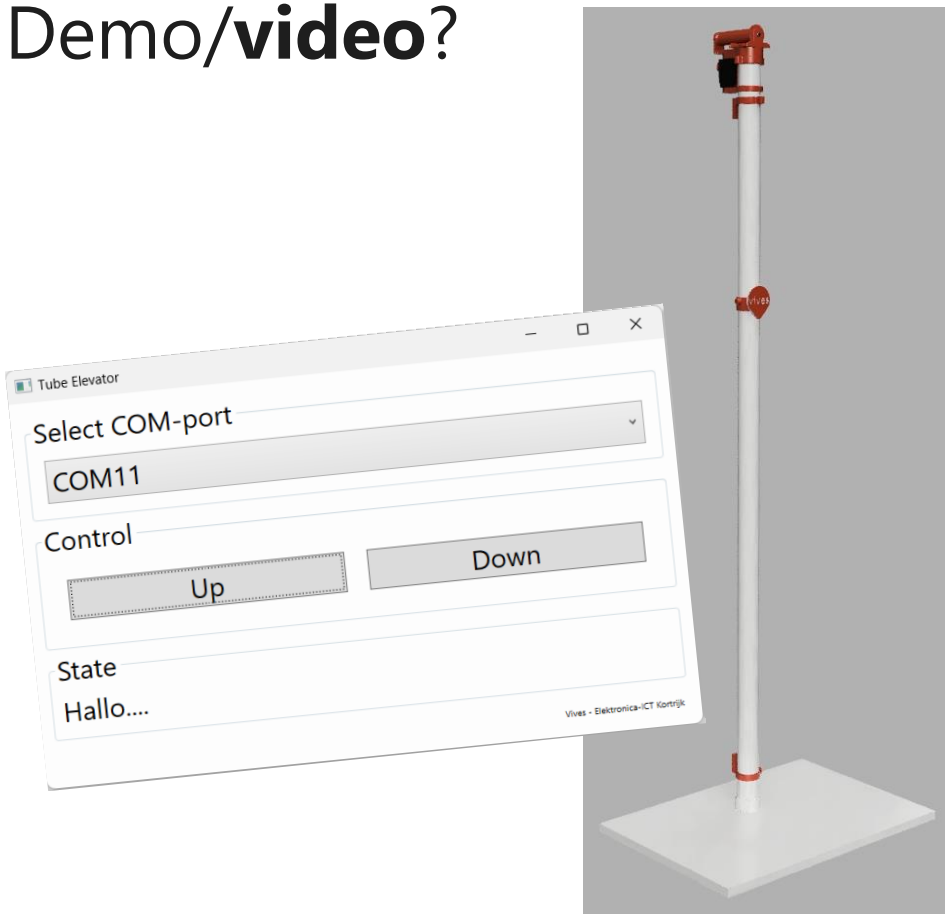
```
if (sp != null && sp.IsOpen)
    sp.Write(new byte[] { data }, 0, 1);
```



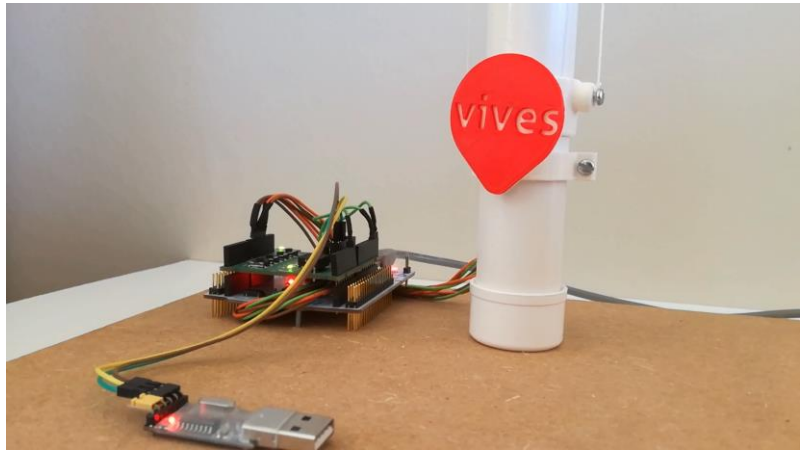
Bron: STM32\ByteToMicrocontroller

Tube Elevator

Demo/**video**?




```
82        case atTop:
83          DisablePWM();
84          ByteToLeds(0x80);
85
86          // Op knop gedrukt of commando via UART binnengekregen...
87          if(SW4Active() || (receivedByteUart == 'D'))
88          {
89            receivedByteUart = 0;
90            nextState = goingDown;
91            ticksStartMovement = 0;
92            sprintf(text, "OK\r\n");
93            StringToUsart1(text);
94            StringToUsart2(text);
95          }
96          break;
```



Bron: https://www.youtube.com/watch?v=1--rh-q7how

Your turn now!

- Is er nog **wat resterende** tijd?

  - Vragen?
  - Feedback voorbije theorieopdrachten?
  - Maak **Theorieopdracht 6 – klokinstellingen.**

  - Ga **zelf aan de slag** om info te verzenden via UART2.
  - Samen opdrachten/oefeningen maken?